

Git & GitHub Guide (VS Code Edition)

A beginner-to-intermediate reference for using Git and GitHub, with an emphasis on VS Code's built-in Source Control tools over raw terminal commands.

1. Why Version Control Exists

The problem before version control

Before VCS, people managed file versions manually:

```
report.doc  
report_final.doc  
report_final_v2.doc  
report_final_v2_ACTUAL.doc  
report_final_USE_THIS_ONE.doc
```

This breaks down fast — no real history, no way to see what changed or why, and it's a nightmare with more than one person editing.

Problems it needed to solve

- **Losing work** — no way to undo a bad change or recover an old version

- **Collaboration conflicts** — two people editing the same file overwrite each other's work
- **No accountability** — no record of who changed what, or why
- **No experimentation safety** — trying something risky meant possibly breaking the only copy

The evolution of version control

1. **Local VCS (1970s–80s)** — tools like RCS kept version history on a single machine, one file at a time. Fine solo, useless for teams.
2. **Centralized VCS (1990s–2000s)** — tools like CVS and Subversion (SVN). One central server holds the only full history; everyone connects to it.
 - *Problem:* if the server goes down, nobody can commit or see history. Single point of failure.
3. **Distributed VCS (DVCS)** — Git (created by Linus Torvalds in 2005, originally to manage the Linux kernel source code). Every developer has a full copy of the entire history on their own machine.
 - No single point of failure
 - You can commit, branch, and view history offline
 - Merging and collaboration are far more flexible

Why Git specifically caught on

- Built for speed and handling huge, fast-moving projects (Linux kernel has thousands of contributors)
- Branching is cheap and fast — encourages experimentation

- Became the de facto standard, and **GitHub (2008)** made hosting/collaborating on Git repos easy for everyone, not just Linux-kernel-level engineers

Bottom line: Git solves the technical problem (tracking changes safely and flexibly), and GitHub solves the social problem (making it easy for people to collaborate around that history).

2. Git vs. GitHub — The Core Difference

	Git	GitHub
What it is	A version control tool that runs on your computer	A website/service that hosts Git repositories online
Adds	Local tracking of file changes over time	Collaboration features (pull requests, issues, project boards)
Dependency	Works without GitHub	Needs Git

Alternatives to GitHub: GitLab, Bitbucket — same idea, different hosts.

Why it matters

- Lets multiple people work on the same code without overwriting each other

- Full history of every change – undo mistakes, see who changed what and why
 - Enables experimentation (branches) without breaking the main project
 - Industry standard – nearly every software job expects Git fluency
-

3. Installation & Setup

Step 1 – Check if Git is already installed

Open VS Code's built-in terminal (`Ctrl+`` / ``Cmd+```):

```
git --version
```

If it shows a version number, Git is installed. If not, install it.

Step 2 – Install Git

- **Windows:** download from git-scm.com → run installer → keep default options
- **Mac:** `brew install git` (or it prompts to install via Xcode tools when you first run `git`)
- **Linux:** `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git` (Fedora)

Verify again with `git --version` .

Step 3 – Install VS Code

Download from code.visualstudio.com and install. Git support is built in – no extension needed for basics.

Step 4 – Configure Git (one-time, identifies your commits)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 5 – Create a GitHub account

Sign up at github.com if you haven't.

Step 6 – Connect your machine to GitHub

Two options – **SSH is recommended**:

SSH method:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

Press Enter through prompts, then copy the public key:

```
cat ~/.ssh/id_ed25519.pub
```

Go to **GitHub** → **Settings** → **SSH and GPG keys** → **New SSH key** → paste it.

HTTPS + token method (simpler, no SSH setup):

GitHub → **Settings** → **Developer settings** → **Personal access tokens** → generate one. You'll paste this as your password when Git asks.

4. VS Code's Source Control Panel (Primary Workflow)

This is how our team should be working day-to-day — using the UI instead of typing commands.

1. Open your project folder in VS Code (`code .` or **File** → **Open Folder**)
2. Click the **branch-looking icon** in the left sidebar (Source Control panel)
3. **Changes** section — lists modified/new files
4. Hover a file → click **+** to stage it (or click **+** next to "Changes" to stage all)
5. Type a commit message in the box at top → click **✓ Commit**
6. Click **Sync Changes** (or the **↑** arrow) to push
7. The bottom-left status bar shows your current branch — click it to **switch/create branches** without typing `git checkout`

Practice it: edit a file, save, then stage → commit → sync using only the panel.

5. First Push Workflow — Creating Your First Repo

Step 1 — Make a project folder

```
mkdir git-practice
cd git-practice
```

```
code .
```

This opens the empty folder in VS Code.

Step 2 — Create a file

In VS Code, create `index.html` with any content:

```
<h1>Hello Git</h1>
```

Save it.

Step 3 — Initialize Git

In VS Code's terminal:

```
git init
```

Step 4 — Stage and commit

Use the **Source Control panel** (Section 4 above), or via terminal:

```
git add .  
git commit -m "Initial commit"
```

Step 5 — Create the GitHub repo

On github.com → **New repository** → name it `git-practice` → don't check "Add README" → **Create**.

Step 6 — Connect and push

GitHub shows you commands — use these in the terminal, or click **Sync Changes** in VS Code once a remote is added:

```
git remote add origin
https://github.com/yourusername/git-practice.git
git branch -M main
git push -u origin main
```

Refresh the GitHub page — your file should be there.

6. Core Terms, Explained Simply

Term	Meaning
Repo (repository)	A project folder tracked by Git
Commit	A saved snapshot of changes
Branch	An independent line of development
Merge	Combining changes from one branch into another
Clone	Copying a remote repo to your local machine
Push	Uploading local commits to the remote (GitHub)
Pull	Downloading and merging remote changes into your local repo
Fork	Your own copy of someone else's repo on GitHub

Term	Meaning
Pull Request (PR)	A request to merge your branch into another, with review
Conflict	When Git can't automatically merge competing changes
.gitignore	A file listing what Git should never track (e.g. <code>node_modules/</code>)

7. Everyday Commands You'll Actually Use

💡 Most of these have VS Code panel equivalents — see Section 4. Terminal versions below for reference.

```
git status          # what's changed, what's
staged
git log             # commit history
git log --oneline   # compact history
git diff           # see exact line changes
before staging
git stash           # temporarily shelve changes
without committing
git stash pop       # bring them back
```

In VS Code: **history and diffs** are visible directly in the Source Control panel and the **Timeline** view at the bottom of the Explorer sidebar — click any file to see its change history without touching the terminal.

8. Undoing Changes (Mistakes Happen)

```
git reset HEAD~1      # undo last commit, keep
changes staged
git revert <commit>    # safely undo a commit
by creating a new "undo" commit (good for shared
branches)
git checkout -- file.txt # discard changes to a
file
```

`reset` **rewrites history** (risky if pushed/shared); `revert` **doesn't** (safer for team branches).

In VS Code, you can also right-click a file in Source Control and choose **Discard Changes** instead of using `checkout --`.

9. Merge vs. Rebase, Cherry-pick, Tags

Merge vs. Rebase

Two ways to bring branch changes together:

- `git merge` — preserves full history, creates a merge commit
- `git rebase` — replays your commits on top of another branch, giving a cleaner linear history (but rewrites commit IDs — avoid on shared branches)

Cherry-pick

Grab a single specific commit from another branch:

```
git cherry-pick <commit-hash>
```

Tags

Mark specific points as releases (e.g. v1.0):

```
git tag v1.0
git push origin v1.0
```

10. Resolving a Merge Conflict in VS Code

Step 1 — Create a conflict on purpose (practice exercise)

On `main`:

```
git checkout -b branch-a
```

Edit `index.html` line 1 to `<h1>Hello from A</h1>`, save, commit:

```
git add .
git commit -m "Change from A"
git checkout main
git checkout -b branch-b
```

Edit the same line to `<h1>Hello from B</h1>`, save, commit:

```
git add .
git commit -m "Change from B"
```

Step 2 — Merge and trigger the conflict

```
git checkout main
git merge branch-a    # merges fine
git merge branch-b    # CONFLICT here
```

Step 3 – Resolve it (in VS Code)

Open `index.html` in VS Code – it now shows:

```
<<<<<<< HEAD
<h1>Hello from A</h1>
=====
<h1>Hello from B</h1>
>>>>>>> branch-b
```

VS Code shows clickable buttons directly above the conflict:

- **Accept Current Change**
- **Accept Incoming Change**
- **Accept Both**
- **Compare**

Click one (or hand-edit), then make sure the `<<<<<<<`, `=====`, `>>>>>>>` markers are all removed.

Step 4 – Finish the merge

```
git add index.html
git commit -m "Resolve conflict between A and B"
```

(Or stage via the Source Control panel and commit with the ✓ button.)

Check `git log --oneline` to see the resolved history.

11. GitHub-Specific Features

- **Issues** — track bugs/tasks/feature requests
- **GitHub Actions** — automate testing/deployment (CI/CD) on push or PR
- **Branch protection rules** — block direct pushes to `main`, require PR reviews
- **GitHub Pages** — host a static website free, straight from a repo
- **README.md** — the front page of your repo, shown automatically
- **LICENSE** — defines how others can use your code
- **.gitattributes** — controls line endings, diff behavior for specific file types
- **Git LFS** — for tracking large files (videos, datasets) that don't belong in normal Git history

12. Typical GitHub Team Workflow

1. `git clone` the repo (or fork it if it's not yours)
2. Create a branch: `git checkout -b fix-bug` (or use the branch switcher in VS Code's status bar)
3. Make changes, then stage + commit (via Source Control panel)
4. Push the branch: **Sync Changes** button, or `git push origin fix-bug`
5. Open a **Pull Request** on GitHub — teammates review and comment

6. Once approved, merge the PR into `main`
 7. Everyone else pulls to get the update (**Sync Changes** or `git pull`)
-

13. Branching Strategy

Git Flow (traditional, more structured)

```
main          → production-ready code
develop       → integration branch
feature/*     → individual features
release/*     → prep for a release
hotfix/*      → urgent production fixes
```

Typical flow:

```
git checkout develop
git checkout -b feature/login-page
# work, commit
git checkout develop
git merge feature/login-page
git branch -d feature/login-page # delete once
merged
```

GitHub Flow (simpler, recommended for most teams)

- Just `main` + short-lived feature branches
- Branch → commit → push → open PR → review → merge
→ delete branch
- No `develop` / `release` branches – simpler, works well for continuous deployment

Recommendation: GitHub Flow is the right fit for most teams, especially ones using GitHub Pages for continuous deployment — branch per change, PR to review, merge to `main`, site auto-updates.

Team workflow concepts

- **Squash merge** — combine all commits in a PR into one clean commit when merging
- **Draft PRs** — mark a PR as "not ready for review" yet
- **Code owners** — auto-assign reviewers based on which files changed

14. GitHub Pages — Hosting a Live Site

Step 1 — Create/prepare the repo

- **User/org site:** repo must be named `username.github.io`
- **Project site:** any repo name works, site lives at `username.github.io/repo-name`

Step 2 — Enable GitHub Pages

On GitHub → your repo → **Settings** → **Pages** (left sidebar):

- **Source:** choose branch (usually `main`) and folder (`/root` or `/docs`)
- **Save**

Site goes live in a minute or two at the URL GitHub shows you.

Step 3 — Clone the repo locally (if not already)

```
git clone https://github.com/username/repo.git
cd repo
code .
```

Step 4 — Add your site files

At minimum an `index.html` in the root (or `/docs` folder, matching what you picked in Step 2).

Step 5 — Preview locally before pushing (optional but recommended)

Use VS Code's **Live Server** extension, or:

```
python3 -m http.server
```

Then open `localhost:8000` to check changes before publishing.

Step 6 — Commit and push changes

Via Source Control panel, or:

```
git add .
git commit -m "Update homepage"
git push origin main
```

Step 7 — See it live

GitHub Pages auto-rebuilds on every push to the configured branch — usually live within 1–2 minutes. Just refresh the site URL.

Ongoing edit cycle (day-to-day loop)

```
# edit files in VS Code
# stage + commit via Source Control panel
# click "Sync Changes" to push
# wait ~1-2 min, refresh browser
```

Optional but useful additions

- **Custom domain:** Settings → Pages → add your domain, plus a CNAME file in the repo
- **Check deploy status:** repo → **Actions** tab shows the "pages build and deployment" run — useful if the site doesn't update, since it shows build errors
- **Jekyll:** GitHub Pages supports Jekyll (static site generator) automatically if you add a `_config.yml` — lets you use templates/layouts instead of raw HTML

Quick Reference — VS Code Panel vs. Terminal

Task	VS Code Panel	Terminal Equivalent
Stage a file	Click + next to file in Source Control	<code>git add file.txt</code>
Stage all	Click + next to "Changes"	<code>git add .</code>
Commit	Type message → click ✓	<code>git commit -m</code>

Task	VS Code Panel	Terminal Equivalent
		"message"
Push	Click Sync Changes / ↑	git push
Pull	Click Sync Changes / ↓	git pull
Switch/create branch	Click branch name in status bar	git checkout - b branch- name
View history	Explorer → Timeline panel	git log
Discard changes	Right-click file → Discard Changes	git checkout - - file.txt
Resolve conflict	Click Accept Current/Incoming/Both buttons	Manually edit conflict markers

Guide compiled for team onboarding — covers setup through deployment, with VS Code's Source Control panel as the primary day-to-day workflow.